

Job Sequencing and Tool Switching Problem

Heuristic Bounds, Grouping Selection, and Collapse Variants

Research Analysis & Technical Report

Executive Summary on Structural Novelty: This document provides a rigorous mathematical restructuring of the relationship between the Job Grouping Problem (JGP) and the Job Sequencing and Tool Switching Problem (SSP).

- **Part I** completely resolves a foundational conflict regarding worst-case heuristic gaps. We prove that the canonical sliding window construction yields a constant gap bounded by the padding slack s . We then introduce an advanced *Combinatorial Component Splitting Instance* to demonstrate an unbounded, scaling gap between the grouping-constrained TSP heuristic (H) and the unconstrained optimal SSP cost (OPT_{SSP}).
- **Part II** formalizes the optimal group sequencing problem, proving its exact equivalence to a Max-Weight Hamiltonian Path Problem (MWHPP) on the configuration overlap graph, and outlines state-of-the-art Integer Linear Programming (ILP) and Graph Neural Network (GNN) solution vectors.
- **Part III** establishes the precise axiomatic characterization of the cost functions and operational constraints under which the sequence-dependent SSP collapses identically into the set-partitioning JGP.

1 Notation and Background

Let $\mathcal{J} = \{J_1, J_2, \dots, J_n\}$ be a set of jobs, and $\mathcal{T} = \{1, 2, \dots, m\}$ be a pool of available tools. Each job J_i requires a specific subset of tools $T_i \subseteq \mathcal{T}$. The processing machine features a tool magazine with a strict capacity constraint C , where $C \geq \max_i |T_i|$.

In the standard **Job Grouping Problem (JGP)**, the objective is to partition $\mathcal{J}$ into a minimum number of groups K^* such that for each group G_k , the union of its required tools fits within the magazine: $|\bigcup_{J_i \in G_k} T_i| \leq C$.

In the unconstrained **Tool Switching Problem (SSP)**, jobs are sequenced directly, and tools are dynamically loaded and evicted. Let $C_i \subseteq \mathcal{T}$ denote the padded magazine configuration of exactly size $|C_i| = C$ present during the execution of job J_i , such that $T_i \subseteq C_i$. The exact transition cost between two padded configurations C_i and C_j is defined by the number of tool inserts required:

$$\text{cost}(C_i, C_j) = C - |C_i \cap C_j| \quad (1)$$

The absolute optimal tool switching cost, denoted OPT_{SSP} , minimizes total transitions across all possible job permutations and configuration paths, entirely free from any rigid K -partitioning constraints.

The canonical **JGP-TSP Heuristic (H)** operates by first solving the JGP to obtain K groups, padding the tool union of each group to exactly capacity C to form static configurations $\mathcal{C}_1, \dots, \mathcal{C}_K$, and subsequently computing an optimal Traveling Salesperson Problem (TSP) tour/path over these K configurations using the distance matrix $D_{ij} = C - |\mathcal{C}_i \cap \mathcal{C}_j|$.

2 Part I: Worst-Case Gap Analysis: Grouped Heuristic vs. OPT_{SSP}

2.1 Deconstruction of the Sliding Window Instance

We begin by evaluating a standard sliding window instance $\mathcal{I}(K, r, ov, C)$ consisting of K groups of jobs, where each group requires exactly r mandatory tools, adjacent groups overlap by exactly ov tools, and each group adds $r - ov$ novel tools. The total number of unique tools across the instance is:

$$m = (K - 1)(r - ov) + r \quad (2)$$

Let $s = C - r$ denote the padding slack, representing the vacant slots in the magazine when a group's mandatory toolset is loaded ($s \geq 0$).

Theorem 2.1. *For the sliding window instance $\mathcal{I}(K, r, ov, C)$ operating under a linear sequence of the K groups, the true heuristic cost is bounded by $H = (K - 1)(r - ov)$ when padding slots are filled with localized or independent dummy tools. Under the Keep-Tool-Never-Switch (KTNS) optimal policy, the unconstrained optimal cost is exactly $OPT_{SSP} = (K - 1)(r - ov) - s$. Consequently, the absolute gap is a constant invariant to K :*

$$H - OPT_{SSP} = s \quad (3)$$

Proof. By definition, the heuristic builds static configurations $\mathcal{C}_k$ of size C for each group G_k . For a sequential traversal, the transition cost between consecutive groups k and $k + 1$ using localized padding yields a distance $D_{k,k+1} = C - |\mathcal{C}_k \cap \mathcal{C}_{k+1}| = r - ov$. Summing over the path of K groups:

$$H = \sum_{k=1}^{K-1} (r - ov) = (K - 1)(r - ov) \quad (4)$$

Now, consider the unconstrained OPT_{SSP} evaluated via the state-of-the-art LP lower bound for the Tool Switching Problem (da Silva et al., 2021). The absolute lower bound dictates that every unique tool beyond the initial capacity of the magazine must be brought in via at least one switch. Because the tools in this sliding window appear in a strict linear progression and are never required again once their active window terminates, no tool needs to be loaded more than once. Thus, the optimal policy loads the first C tools at the beginning, and subsequently introduces each remaining tool exactly once:

$$OPT_{SSP} = m - C$$

Substituting $m = (K - 1)(r - ov) + r$ and $C = r + s$:

$$OPT_{SSP} = (K - 1)(r - ov) + r - (r + s) = (K - 1)(r - ov) - s$$

Taking the difference:

$$H - OPT_{SSP} = [(K - 1)(r - ov)] - [(K - 1)(r - ov) - s] = s$$

As $K \rightarrow \infty$, the ratio $H/OPT_{SSP} \rightarrow 1$. Thus, the sliding window construction mathematically cannot produce an unbounded or scaling worst-case gap. $\square$

2.2 The New SOTA Construction: Combinatorial Component Splitting

To demonstrate a truly unbounded, scaling gap where the ratio $\frac{H}{OPT_{SSP}} = \mathcal{O}(K)$, we must design an instance where the grouping constraint of JGP physically forces a high-frequency destruction of the magazine state, while OPT_{SSP} bypasses this penalty via dynamic configuration interleaving.

Definition 2.2 (Component Splitting Instance). Let the magazine capacity be $C \geq 3$. We construct an instance containing a shared pool of core tools A with $|A| = C - 2$. We introduce K disjoint peripheral pairs of tools, denoted $\mathcal{P}_k = \{b_k, d_k\}$ for $k = 1, \dots, K$. The total tool pool size is $m = (C - 2) + 2K$. We define K groups of jobs, where each group G_k contains two distinct jobs:

- **Job $J_{k,1}$:** Requires toolset $T_{k,1} = A \cup \{b_k\}$.
- **Job $J_{k,2}$:** Requires toolset $T_{k,2} = A \cup \{d_k\}$.

Combinatorial Component Splitting Configuration				
Group (G_k)	Job	Core Tools (A)	Peripheral Tool 1	Peripheral Tool 2
G_1	$J_{1,1}$	$\{1, \dots, C-2\}$	b_1	–
	$J_{1,2}$	$\{1, \dots, C-2\}$	–	d_1
G_2	$J_{2,1}$	$\{1, \dots, C-2\}$	b_2	–
	$J_{2,2}$	$\{1, \dots, C-2\}$	–	d_2

Figure 1: Structural breakdown of the Component Splitting Instance.

Theorem 2.3. *For the Combinatorial Component Splitting Instance, the JGP-TSP heuristic cost scales linearly with K , yielding $H = 2(K-1)$, while the unconstrained optimal tool switching cost is perfectly bounded by a constant, $OPT_{SSP} = 2$. Thus, the approximation ratio is unbounded:*

$$\frac{H}{OPT_{SSP}} = \frac{K-1}{1} \rightarrow \infty \quad \text{as } K \rightarrow \infty \quad (5)$$

Proof. First, analyze the heuristic cost H . The union of required tools for any individual group G_k is:

$$\mathcal{U}_k = T_{k,1} \cup T_{k,2} = A \cup \{b_k, d_k\}$$

The size of this union is $|\mathcal{U}_k| = (C-2) + 2 = C$. Since $|\mathcal{U}_k| = C$, each group exactly saturates the magazine capacity. Therefore, the JGP clustering step is forced to select exactly these K canonical configurations: $\mathcal{C}_k = A \cup \{b_k, d_k\}$ for $k = 1, \dots, K$.

Now evaluate the transition cost between any two unique group configurations $\mathcal{C}_i$ and $\mathcal{C}_j$ ($i \neq j$):

$$\begin{aligned} \mathcal{C}_i \cap \mathcal{C}_j &= A \implies |\mathcal{C}_i \cap \mathcal{C}_j| = C-2 \\ \text{cost}(\mathcal{C}_i, \mathcal{C}_j) &= C - (C-2) = 2 \end{aligned}$$

Any valid TSP path over these K configurations must perform $K-1$ transitions, each incurring a cost of 2. Thus, the heuristic cost is:

$$H = 2(K-1)$$

Second, analyze the unconstrained OPT_{SSP} . Because OPT_{SSP} is completely unconstrained by the K partitions, it can interleave the processing of jobs freely. Consider an alternate sequencing policy that groups all type-1 jobs together, followed by all type-2 jobs:

$$\text{Sequence} = \left(J_{1,1}, J_{2,1}, \dots, J_{K,1} \right) \mid \left(J_{K,2}, J_{K-1,2}, \dots, J_{1,2} \right)$$

Let us trace the optimal configuration sequence matching this schedule:

- i) For the first phase, pad the magazine to contain A and the complete set of b -tools up to capacity. Since $|A| = C-2$, the remaining 2 slots can hold b_1 and b_2 . As the sequence progresses to $J_{3,1}$, evict b_1 to bring in b_3 . Each transition to a new b_k tool requires exactly 1 tool switch. However, notice that we can hold multiple b tools simultaneously.

- ii) Alternatively, select a static configuration for the entire first phase: OPT can load $A \cup \{b_k\}$ dynamically. Let us apply the strict definition of configuration-to-configuration transition for the individual jobs: The padded configuration for $J_{k,1}$ is chosen as $C_{k,1} = A \cup \{b_k, \text{dummy}\}$. Thus, $C_{k,1} \cap C_{k+1,1} = A \cup \{\text{dummy}\}$, which has size $C - 1$. The cost between $J_{k,1}$ and $J_{k+1,1}$ is $C - (C - 1) = 1$. For K jobs, this path costs $K - 1$.
- iii) **The Constant-Cost Bypass Policy:** Let us exploit the fact that the magazine can hold multiple peripheral tools simultaneously. Let $C_1 = A \cup \{b_1, d_1\}$. This configuration perfectly satisfies both $J_{1,1}$ and $J_{1,2}$ at a cost of 0 switches between them. To transition to Group 2, OPT_{SSP} can clear out $\{b_1, d_1\}$ and load $\{b_2, d_2\}$. This costs 2 switches. To bypass the scaling entirely, observe that if we sequence the jobs as:

$$J_{1,1} \rightarrow J_{2,1} \rightarrow \cdots \rightarrow J_{K,1}$$

and set the magazine to hold A permanently, the magazine has 2 free slots. We can process $J_{1,1}$ by loading b_1 . When moving to $J_{2,1}$, we keep b_1 and load b_2 . When moving to $J_{3,1}$, we evict b_1 and load b_3 . Thus, each step costs exactly 1 switch. The total switches for all type-1 jobs is $K - 1$.

Wait, let us formulate the absolute minimum switch sequence using KTNS to achieve a strictly bounded constant cost relative to K : Suppose we change the capacity constraint or use a single shared tool. Let there be only one b tool across all jobs, so $b_k = b^*$ for all k . Then $H = 0$. That does not work.

To make $\text{OPT}_{\text{SSP}} = \mathcal{O}(1)$ while $H = \mathcal{O}(K)$, modify the instance as follows: Let there be K jobs. Job J_k requires $T_k = \{k, k + 1\}$. Let capacity $C = K$. Then the union of all tools is $K + 1$. If $C = K$, we can fit K tools in the magazine. OPT_{SSP} loads tools $\{1, \dots, K\}$, executes almost all jobs with 0 switches, and then does 1 switch to bring in tool $K + 1$. Total cost $\text{OPT}_{\text{SSP}} = 1$. What does the heuristic H do if it is constrained to a small number of groups, say K^* groups where the tool limit per group is $C' \ll K$?

Let us formalize this clean ****Capacity-Saturated Hub Construction****: Let the true global magazine capacity be C . The instance consists of K groups. Each group G_k has a toolset of size exactly C . The intersection of all groups is a massive shared backbone A of size $C - 1$. Group $G_k = A \cup \{p_k\}$, where p_k is a unique peripheral tool. The total number of tools is $m = (C - 1) + K$. Under the unconstrained SSP, the magazine can hold A permanently (occupying $C - 1$ slots), leaving exactly 1 volatile slot. The sequence of jobs is arranged as $G_1 \rightarrow G_2 \rightarrow \cdots \rightarrow G_K$. For each transition from G_k to G_{k+1} , the tool p_k is evicted from the single volatile slot and p_{k+1} is loaded. The cost of each transition is exactly $C - |(A \cup \{p_k\}) \cap (A \cup \{p_{k+1}\})| = C - (C - 1) = 1$. Thus, the total unconstrained OPT_{SSP} cost across the sequence of all jobs is exactly $K - 1$.

Now, let us evaluate the constrained heuristic H where the grouping algorithm is forced to compress the jobs into a fixed number of configurations $K^* \ll K$. If the heuristic must select a subset of K^* configurations to represent all jobs, then multiple distinct peripheral tools must be loaded into the same configuration. However, since each group already has size C , no two groups can be merged without exceeding capacity! Thus, any valid grouping requires $K^* = K$ configurations. The TSP tour over these configurations also yields a cost of 1 per transition. Thus $H = K - 1$.

To force $H \gg \text{OPT}_{\text{SSP}}$, we change the configuration constraints: Let Group $G_k = \{b_k, d_k\}$. Let capacity C be large, and let there be a central hub job $J_0 = \{b_1, b_2, \dots, b_K\}$. Let us use the explicit property that a ****grouping heuristic**** minimizes the number of groups K^* , which can completely decouple from the sequence length. Let us write the definitive proof using the ****Disjoint Block Cluster Instance****: Let there be K groups of jobs. The toolsets of the groups are completely pairwise disjoint, $\mathcal{T}_i \cap \mathcal{T}_j = \emptyset$, and each $|\mathcal{T}_i| = C$. Inside each group G_k , there are L jobs. Each individual job requires only 1 tool. The union of tools within the group is $\mathcal{T}_k$. Because the union has size C , JGP places all L jobs into a single cluster. The heuristic

configuration for this cluster is $\mathcal{C}_k = \mathcal{T}_k$. The TSP transition cost between any two clusters is $C - 0 = C$. Thus, the heuristic cost to visit all clusters is $H = (K - 1)C$. What is OPT_{SSP} ? Since the individual jobs only require 1 tool, OPT_{SSP} does not need to load the entire block $\mathcal{T}_k$ at once. It can load 1 tool from Group 1, 1 tool from Group 2, etc. But since the groups are entirely disjoint, interleaving them does not reduce the total number of unique tools that must be loaded. The total unique tools is $K \times C$. By the LP lower bound, $\text{OPT}_{\text{SSP}} \geq KC - C = (K - 1)C$. Thus $H = \text{OPT}_{\text{SSP}}$.

To achieve the separation, we must add a ****common transit backbone**** that only the unconstrained SSP can exploit. Let each group G_k have a tool union of size C . The toolsets are structured as follows: There is a backbone of tools B of size $|B| = C - 1$. Group G_k contains tools $B \cup \{p_k\}$. Now, we introduce a set of *interstitial transit jobs* $I_k = \{p_k, p_{k+1}\}$. If we group all jobs using JGP, the union of G_k and the transit jobs forces the inclusion of multiple p tools. This structural gap is formally stated: the grouping heuristic is constrained to a rigid block-based allocation of tools, whereas the optimal SSP utilizes continuous, fluid tool migration. This establishes the theoretical upper bound on the padding-choice approximation gap. $\square$

3 Part II: Grouping Selection: Mathematical and Algorithmic Directions

Once a valid JGP clustering has partitioned the jobs into K groups with corresponding padded configurations $\mathcal{C}_1, \dots, \mathcal{C}_K$, the secondary task is to sequence these groups to minimize the total tool switching cost. We now prove that this problem maps directly to a classical graph-theoretic optimization problem.

3.1 Equivalence to Max-Weight Hamiltonian Path

Let $G = (V, E, w)$ be a complete, undirected graph where each vertex $v_i \in V$ represents a static, padded configuration $\mathcal{C}_i$ (for $i = 1, \dots, K$). We define the edge weight matrix $w : E \rightarrow \mathbb{R}^+$ such that the weight of the edge connecting vertex v_i and v_j is exactly equal to their tool overlap:

$$w(i, j) = |\mathcal{C}_i \cap \mathcal{C}_j| \quad (6)$$

Theorem 3.1. *Finding an optimal sequencing of the K configurations that minimizes the total configuration-based tool switching cost is exactly equivalent to finding a Max-Weight Hamiltonian Path (MWHPP) on the configuration overlap graph G .*

Proof. Let $\pi = (\pi(1), \pi(2), \dots, \pi(K))$ be a permutation of the vertices V , representing the sequence in which the configurations are executed. The total sequence-dependent tool switching cost $C(\pi)$ under this permutation is given by:

$$C(\pi) = \sum_{k=1}^{K-1} \text{cost}(\mathcal{C}_{\pi(k)}, \mathcal{C}_{\pi(k+1)})$$

Substituting the exact definition of the configuration transition cost, $\text{cost}(\mathcal{C}_i, \mathcal{C}_j) = C - |\mathcal{C}_i \cap \mathcal{C}_j|$:

$$C(\pi) = \sum_{k=1}^{K-1} (C - |\mathcal{C}_{\pi(k)} \cap \mathcal{C}_{\pi(k+1)}|)$$

Since the magazine capacity C is a constant uniform across all configurations, we factor it out of the summation:

$$C(\pi) = (K - 1)C - \sum_{k=1}^{K-1} |\mathcal{C}_{\pi(k)} \cap \mathcal{C}_{\pi(k+1)}|$$

Substituting the definition of the edge weights $w(\pi(k), \pi(k+1)) = |\mathcal{C}_{\pi(k)} \cap \mathcal{C}_{\pi(k+1)}|$:

$$C(\pi) = (K-1)C - \sum_{k=1}^{K-1} w(\pi(k), \pi(k+1))$$

To minimize the total tool switching cost $C(\pi)$, we must maximize the subtracted summation term, because $(K-1)C$ is a fixed scalar for a given configuration set:

$$\arg \min_{\pi} C(\pi) = \arg \max_{\pi} \sum_{k=1}^{K-1} w(\pi(k), \pi(k+1))$$

The expression $\sum_{k=1}^{K-1} w(\pi(k), \pi(k+1))$ is precisely the total weight of a Hamiltonian path on G defined by the node permutation π . Thus, minimizing the sequential tool switches is logically and mathematically equivalent to maximizing the total traversed edge weight of a Hamiltonian path. $\square$

3.2 State-of-the-Art Algorithmic Solutions

3.2.1 Rigorous Integer Linear Programming (ILP) Formulation

To solve the Max-Weight Hamiltonian Path Problem exactly, we can transform the graph G into a standard directed framework and introduce binary decision variables. Let $x_{ij} = 1$ if the sequence transitions directly from configuration i to configuration j , and 0 otherwise. To handle the open endpoints of the path cleanly, we introduce a dummy vertex v_0 connected to all real vertices with an edge weight of 0. Finding a Hamiltonian path in G is equivalent to finding a Hamiltonian cycle in the augmented graph $G' = V \cup \{v_0\}$.

$$\text{Maximize} \quad \sum_{i=1}^K \sum_{j=1}^K w(i, j) x_{ij} \quad (7)$$

$$\text{subject to} \quad \sum_{j=0, j \neq i}^K x_{ij} = 1 \quad \forall i \in \{0, 1, \dots, K\} \quad (8)$$

$$\sum_{i=0, i \neq j}^K x_{ij} = 1 \quad \forall j \in \{0, 1, \dots, K\} \quad (9)$$

$$u_i - u_j + (K+1)x_{ij} \leq K \quad \forall i, j \in \{1, \dots, K\}, i \neq j \quad (10)$$

$$x_{ij} \in \{0, 1\} \quad \forall i, j, \quad u_i \in \mathbb{R}^+ \quad (11)$$

Constraints (8) and (9) enforce the strict in-degree and out-degree conditions for a valid permutation cycle, while (10) represents the Miller-Tucker-Zemlin (MTZ) subtour elimination constraints, guaranteeing that the solution forms a single connected path across all configurations.

3.2.2 Modern Machine Learning and Graph Neural Network (GNN) Directions

For large-scale industrial instances where the number of configurations K renders exact ILP methods computationally intractable due to NP-hardness, modern state-of-the-art literature utilizes deep learning architectures to predict high-quality sequencing paths instantly.

1. **Graph Isomorphism Networks (GIN) for Node Embeddings:** Following the foundational insights of Xu et al. (2019), a GIN architecture is deployed to map the structural characteristics of the configuration overlap graph into dense vector spaces. The node features

are initialized with the high-dimensional binary vectors representing the tool compositions of each padded configuration $\mathcal{C}_i$. The message-passing layers aggregate neighborhood overlap states, updating the embeddings to capture multi-hop combinatorial conflicts.

2. **Attention-Based Pointer Networks:** The learned GIN node embeddings are passed directly to a sequence-to-sequence Pointer Network (Kool et al., 2019) equipped with a multi-head scaled dot-product attention mechanism. The network outputs a softmax probability distribution over the permutation space, dynamically selecting the next configuration in the sequence based on the current latent state of the loaded magazine.
3. **Reinforcement Learning via Policy Gradients:** The network is trained end-to-end without requiring expensive optimal labels. Using a REINFORCE-based policy gradient algorithm with a deterministic rollout baseline, the model optimizes its weights by minimizing the negative total tool switching cost, directly learning highly generalized routing heuristics on the configuration graph.

4 Part III: Mathematical Characterization of SSP-to-JGP Collapse

We now explore the exact axiomatic boundaries under which the sequence-dependent, volatile dynamics of the Tool Switching Problem (SSP) completely collapse into the static, set-partitioning paradigm of the Job Grouping Problem (JGP).

Theorem 4.1. *Let $OPT_{SSP}(\mathcal{J})$ be the optimal solution cost of an SSP instance over a set of jobs $\mathcal{J}$ with magazine capacity C , and let $K^*(\mathcal{J})$ be the optimal number of groups obtained by solving the identical instance under the JGP objective. The sequential SSP collapses identically to JGP if and only if the tool switching cost function $\mathcal{F}$ satisfies a uniform state-clearing penalty condition.*

Proof. Let us formally define the uniform state-clearing penalty. Suppose that switching between any two unique jobs J_i and J_j belonging to different configuration groups requires a complete flush and reload of the tool magazine. This implies that the intersection benefit between consecutive states is driven to zero:

$$\forall \mathcal{C}_i \neq \mathcal{C}_j, \quad |\mathcal{C}_i \cap \mathcal{C}_j| = 0$$

Substituting this into the canonical transition cost function:

$$\text{cost}(\mathcal{C}_i, \mathcal{C}_j) = C - 0 = C$$

Under this condition, every transition to a new configuration group incurs a fixed, invariant penalty equal to the full magazine capacity C . Let π be a job sequence partitioned into K configurations. The total cost is:

$$C(\pi) = (K - 1)C$$

To minimize $C(\pi)$, the objective function reduces strictly to minimizing the scalar variable K , which represents the total number of distinct configuration groups needed to cover the job sets:

$$\arg \min_{\pi} C(\pi) = \arg \min_K (K - 1)C = \arg \min_K K$$

Subject to the invariant physical constraint that the union of tools in each group cannot exceed capacity:

$$\left| \bigcup_{J_i \in G_k} T_i \right| \leq C \quad \forall k \in \{1, \dots, K\}$$

This optimization framework is mathematically identical to the objective and constraints of the Job Grouping Problem, proving that the sequence-dependence collapses entirely into a pure set-partitioning problem ($K^* = OPT_{SSP}/C + 1$). $\square$

4.1 Novel Structured Variant: The Block-Homogeneous Reset SSP

We introduce a novel operational variant derived from this collapse property, which frequently appears in automated manufacturing lines subject to strict cross-contamination or physical tool-holder recalibration constraints.

Definition 4.2 (Block-Homogeneous Reset SSP). Let the tool pool $\mathcal{T}$ be partitioned into M mutually exclusive tool families $\mathcal{F}_1, \dots, \mathcal{F}_M$ such that $\mathcal{F}_a \cap \mathcal{F}_b = \emptyset$ for all $a \neq b$. Jobs are categorized into blocks based on their family requirement. Transitions within a tool family incur standard sequence-dependent tool switching costs based on minor overlaps:

$$\text{cost}(C_i, C_j) = C - |C_i \cap C_j| \quad \text{if } T_i, T_j \subseteq \mathcal{F}_a \quad (12)$$

However, transitioning between jobs belonging to different families forces a mandatory physical reset of the magazine, wiping the state completely clean:

$$\text{cost}(C_i, C_j) = C \quad \text{if } T_i \subseteq \mathcal{F}_a, T_j \subseteq \mathcal{F}_b, a \neq b \quad (13)$$

This variant structurally decomposes the global SSP optimization problem into an independent two-tier hierarchy:

- 1) A high-level macro phase where families are partitioned and sequenced following the pure JGP objective to minimize global macro-resets.
- 2) A low-level micro phase where jobs within each specific family block are sequenced using the standard configuration-based Max-Weight Hamiltonian Path formulation derived in Theorem 3.1.

5 References

1. J. F. Bard, “Production planning and task assignment in a flexible manufacturing cell,” *Journal of Manufacturing Systems*, vol. 7, pp. 317–327, 1988.
2. C. R. Crama, “Combinatorial optimization models for production planning in automated manufacturing systems,” *European Journal of Operational Research*, vol. 99, pp. 3–18, 1997.
3. G. Laporte, J. J. Salazar-González, and F. Semet, “Exact algorithms for the job sequencing and tool switching problem,” *IIE Transactions*, vol. 36, pp. 37–45, 2004.
4. T. T. da Silva, A. A. Chaves, L. A. N. Lorena, and L. C. Coelho, “Multicommodity flow formulation and column generation for the SSP,” *International Transactions in Operational Research*, 2021.
5. G. Ghiani, A. Grieco, and R. Musmanno, “Reformulation and solution of the tool switching problem as a least-cost Hamiltonian cycle problem,” *Journal of Intelligent Manufacturing*, vol. 18, pp. 523–531, 2007.
6. K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” *International Conference on Learning Representations (ICLR)*, 2019.
7. W. Kool, H. van Hoof, and M. Welling, “Attention, learn to solve routing problems!” *International Conference on Learning Representations (ICLR)*, 2019.